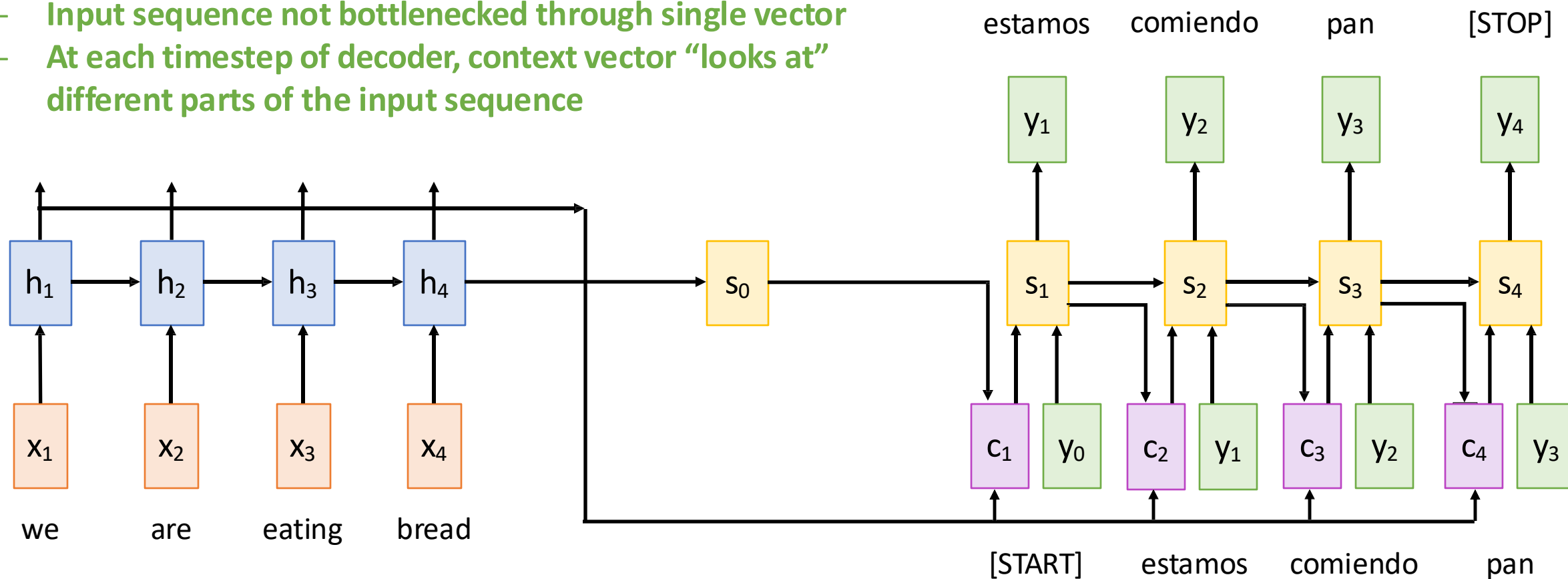


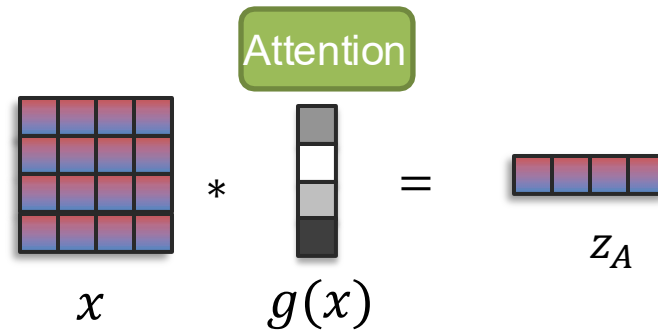
Sequence-to-Sequence with RNNs and Attention

Use a different context vector in each timestep of decoder

- Input sequence not bottlenecked through single vector
- At each timestep of decoder, context vector “looks at” different parts of the input sequence



Last time: Attentive Pooling



$$z_A = x_A * g(x) = \sum [g(x)]_i \cdot x_i; \quad \begin{array}{l} x \in \mathbb{R}^d \times n \\ g(x) \in n \times 1 \end{array}$$

Attention Layer

Inputs:

Query vectors: $\mathbf{Q}$ (Shape: $N_Q \times D_Q$)

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Computation:

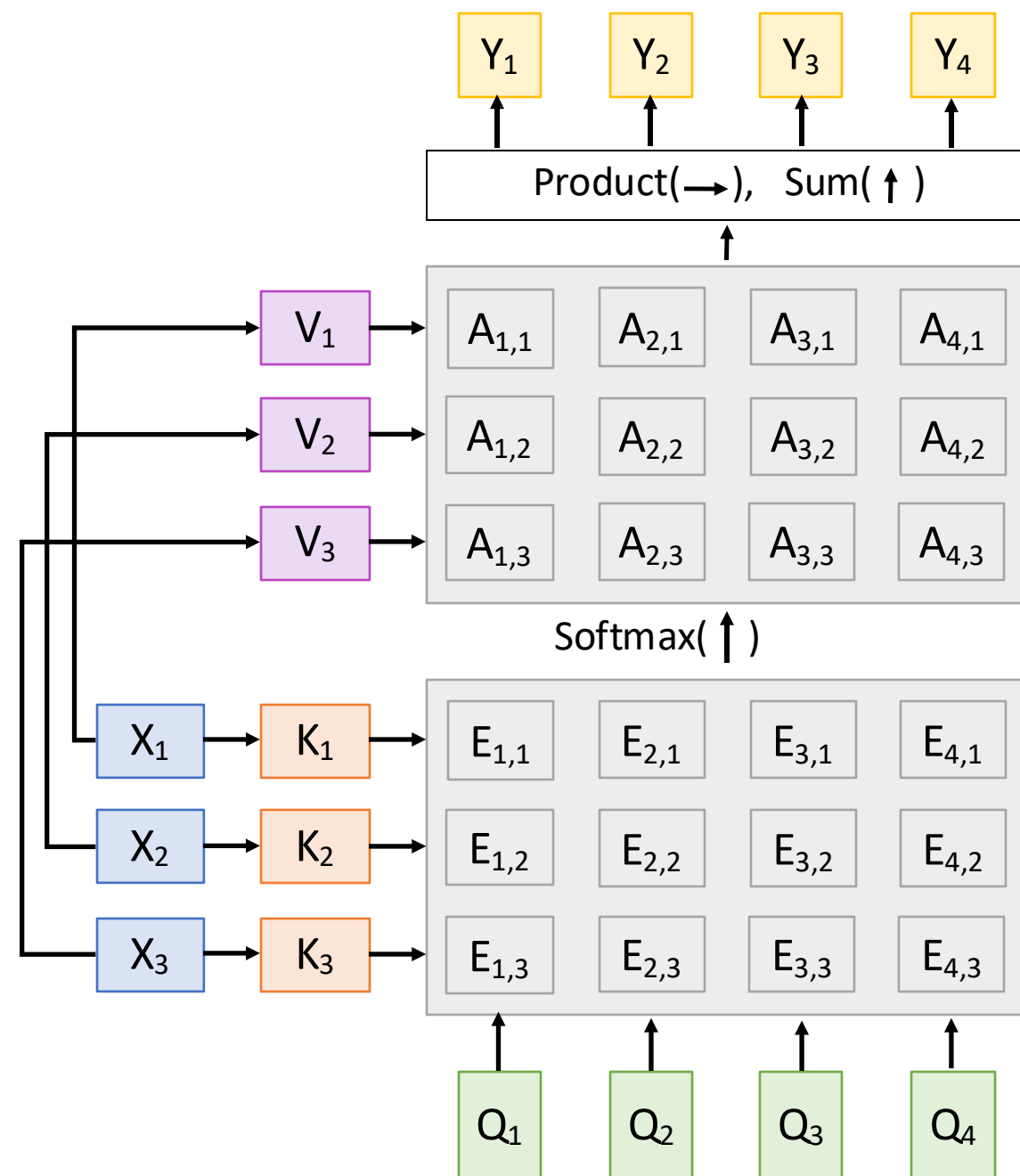
Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_Q \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_Q \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_Q \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$



Self-Attention Layer

One **query** per **input vector**

- Inputs:

- Input vectors: X (Shape: $N_x \times D_x$) Key matrix: W_K (Shape: $D_x \times D_Q$) Value matrix: W_V (Shape: $D_x \times D_V$) Query matrix: W_Q (Shape: $D_x \times D_Q$)

- Computation:

- Query vectors: $Q = XW_Q$

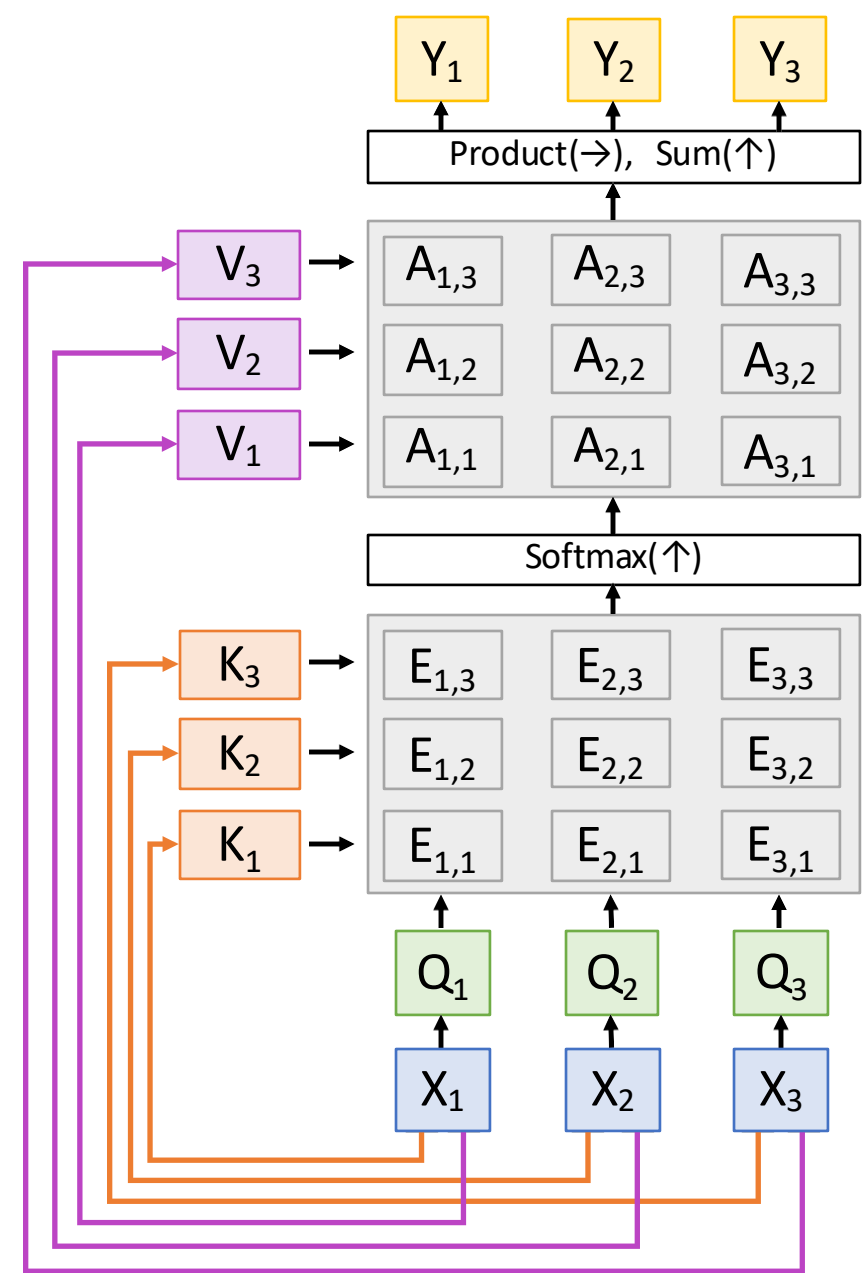
- Key vectors: $K = XW_K$ (Shape: $N_x \times D_Q$)

- Value Vectors: $V = XW_V$ (Shape: $N_x \times D_V$)

- Similarities: $E = QK^T$ (Shape: $N_x \times N_x$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$

Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_x \times N_x$) Output vectors:

$Y = AV$ (Shape: $N_x \times D_V$) $Y_i = \sum_j A_{i,j} V_j$



Permutation Equivariant

$$f(s(x)) = s(f(x))$$

Inputs:

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Query matrix: $\mathbf{W}_Q$ (Shape: $D_X \times D_Q$)

Computation:

Query vectors: $\mathbf{Q} = \mathbf{XW}_Q$

Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

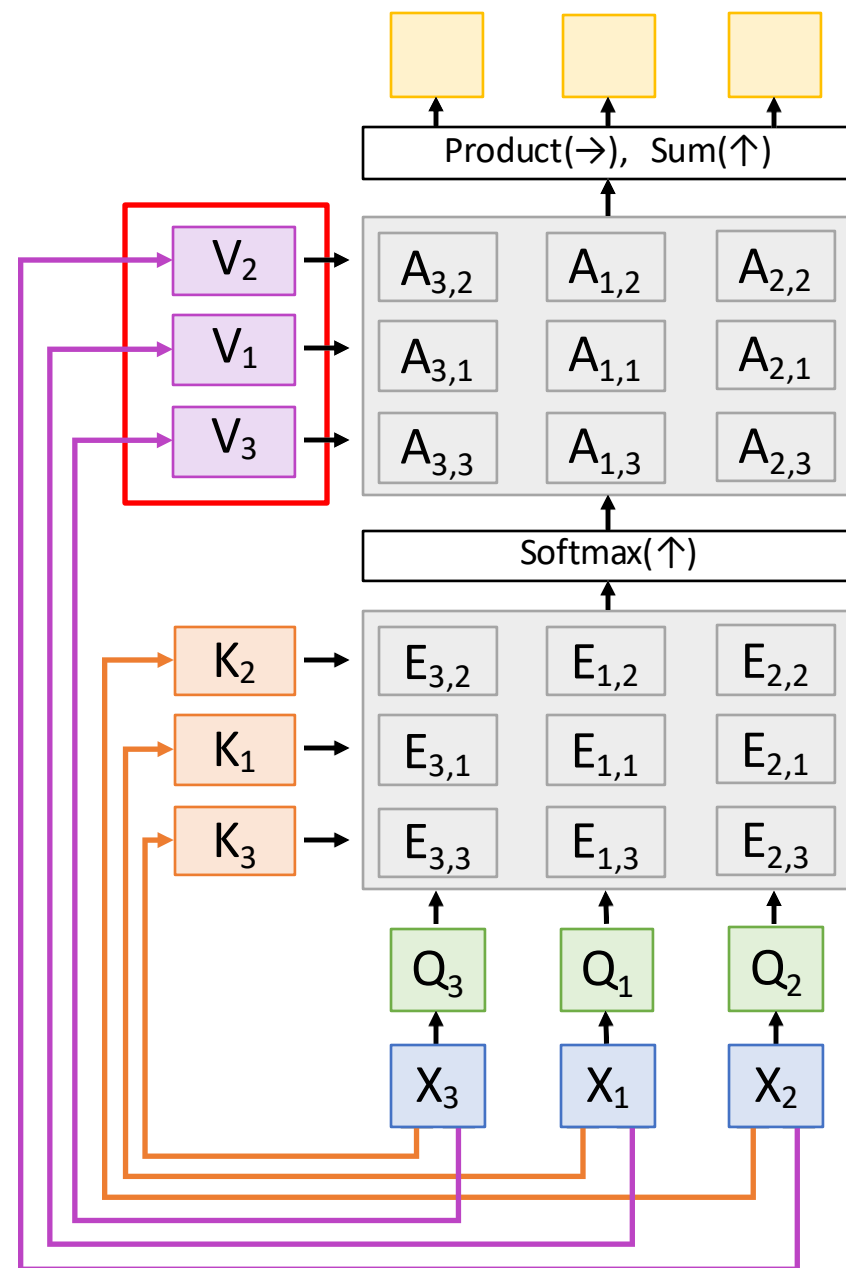
Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_X \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_X \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$

Consider **permuting**
the input vectors:

Values will be the
same, but **permuted**



Masked Self-Attention Layer

Don't let vectors "look ahead" in the sequence
Used for language modeling (predict next word)

Inputs:

Input vectors: $\mathbf{X}$ (Shape: $N_x \times D_x$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_x \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_x \times D_V$)

Query matrix: $\mathbf{W}_Q$ (Shape: $D_x \times D_Q$)

Computation:

Query vectors: $\mathbf{Q} = \mathbf{XW}_Q$

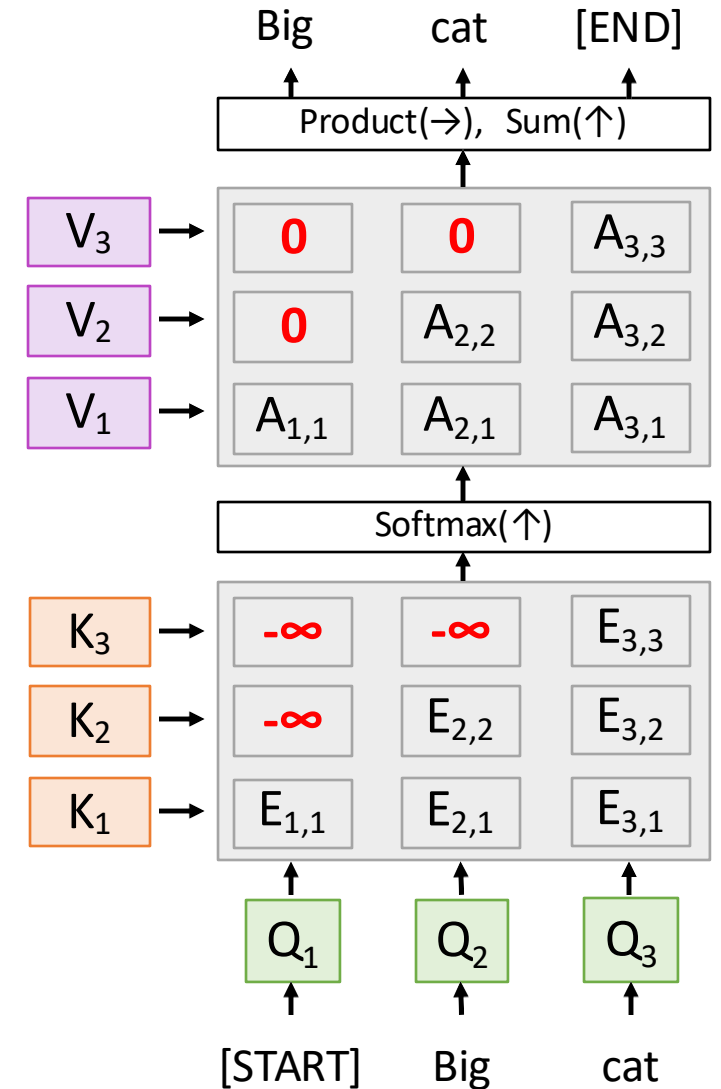
Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_x \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_x \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_x \times N_x$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_x \times N_x$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_x \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$



Multihead Self-Attention Layer

Use H independent
“Attention Heads” in parallel

Inputs:

Input vectors: $\mathbf{X}$ (Shape: $N_x \times D_x$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_x \times D_K$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_x \times D_V$)

Query matrix: $\mathbf{W}_Q$ (Shape: $D_x \times D_Q$)

Computation:

Query vectors: $\mathbf{Q} = \mathbf{XW}_Q$

Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_x \times D_K$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_x \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_x \times N_x$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

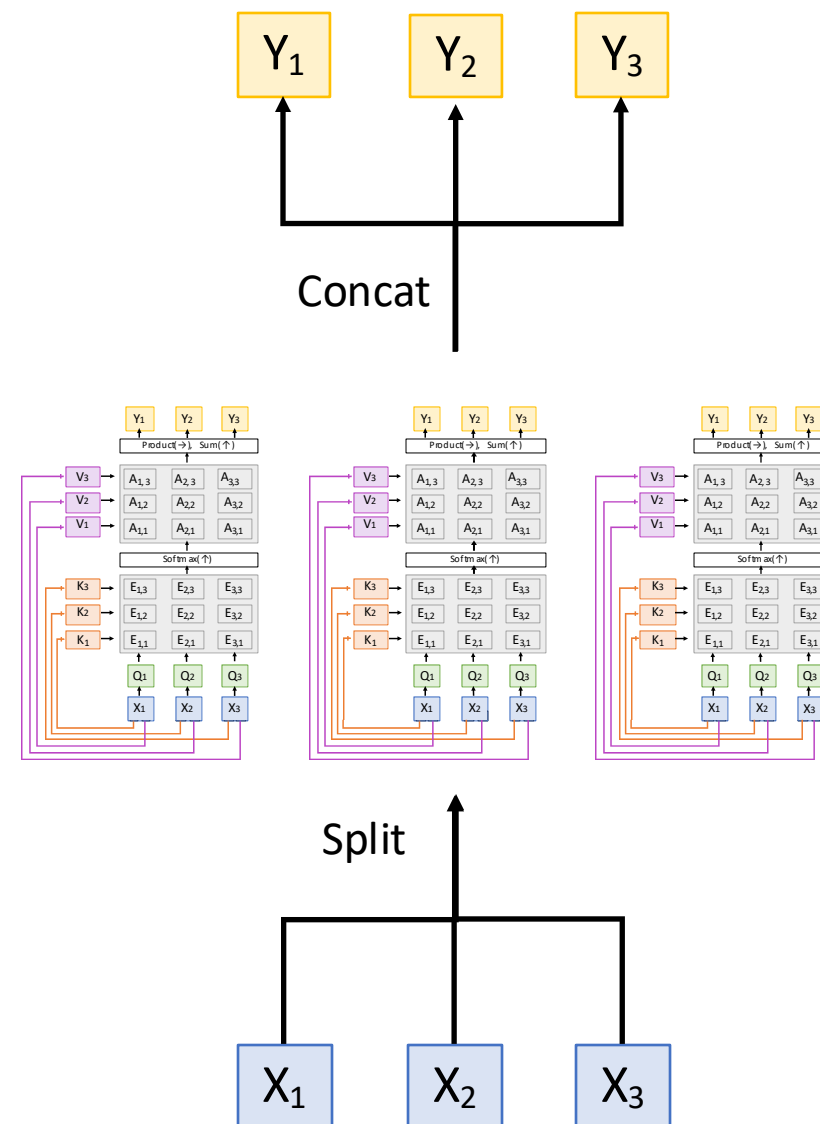
Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_x \times N_x$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_x \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$

Hyperparameters:

Query dimension D_Q

Number of heads H



The Transformer

Transformer Block:

Input: Set of vectors x

Output: Set of vectors y

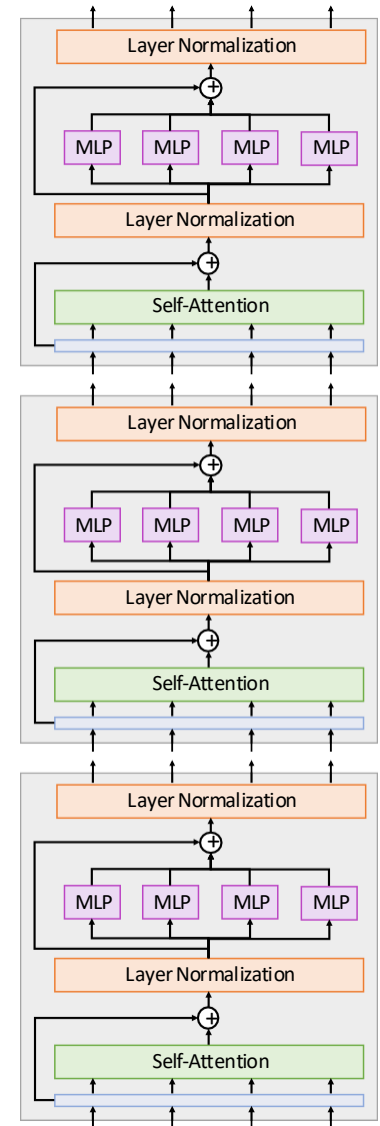
Self-attention is the only
interaction between vectors!

Layer norm and MLP work
independently per vector

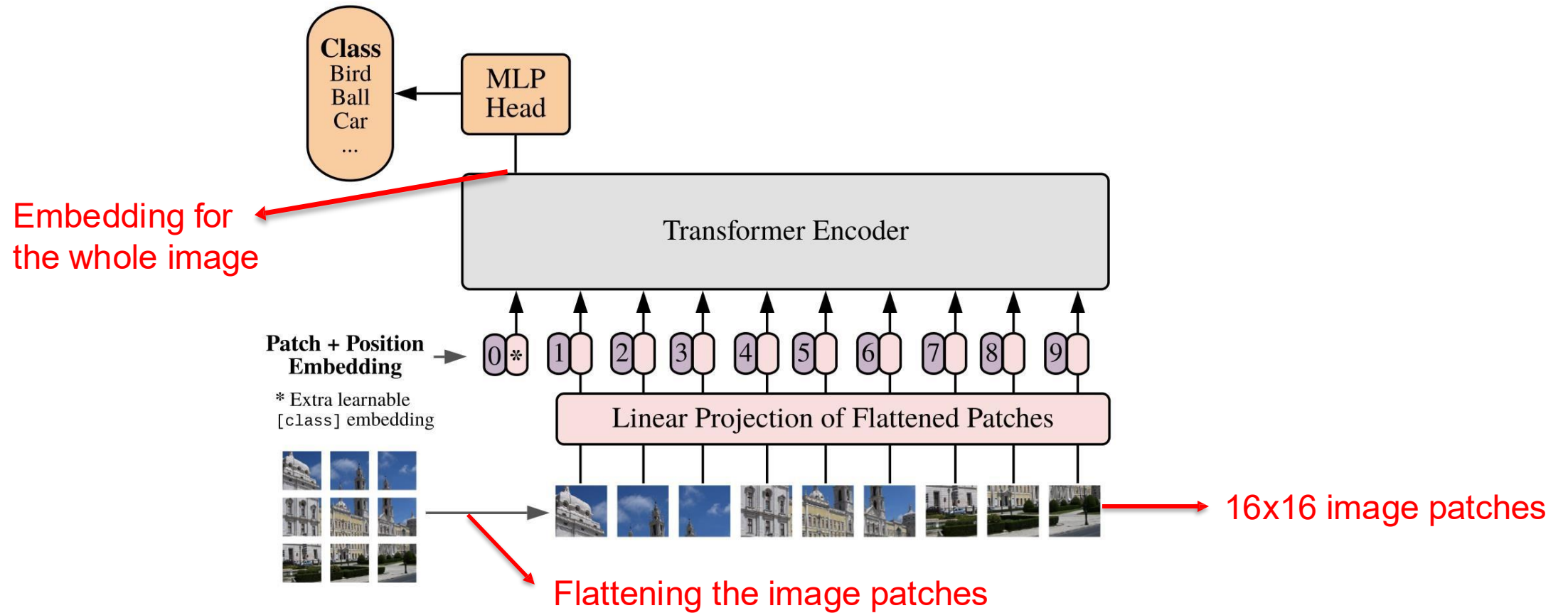
Highly scalable, highly
parallelizable

A **Transformer** is a sequence
of transformer blocks

Vaswani et al:
12 blocks, $D_Q=512$, 6 heads

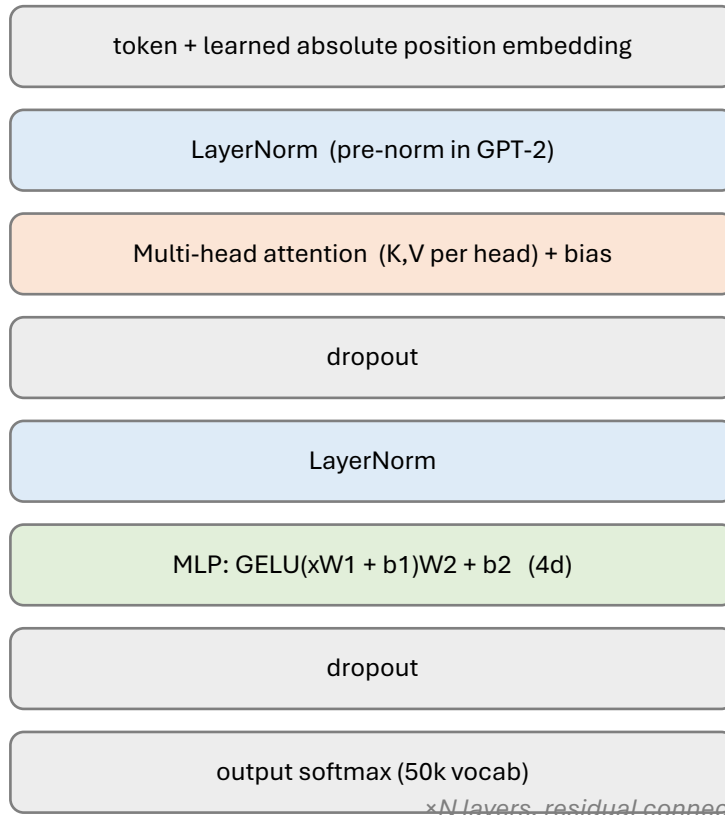


Vision Transformer

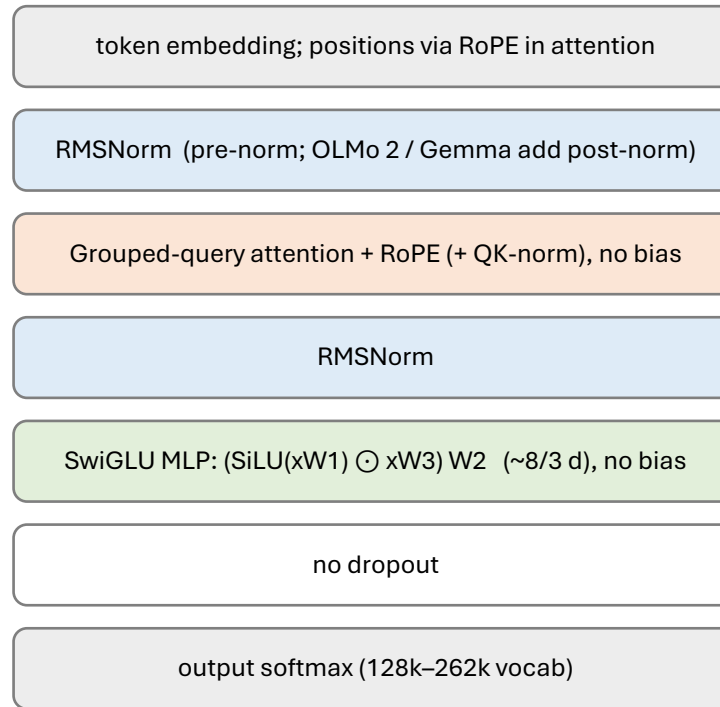


The block you learned vs. the block used today

GPT-2 block (2019)



Llama-3 / Qwen3-style block (2024–25)



- Same skeleton: attention + MLP with residual connections
- Every part swapped for a cheaper or more stable variant; gone: dropout, biases, the learned position table
- Vocabulary 50k → 128k (Llama 3) – 262k (Gemma 3)

×N layers, residual connections around attention and MLP in both

Why each change? (one line each)

- **RMSNorm** (Zhang & Sennrich, 2019): drop LayerNorm's mean-centering and bias — cheaper, same quality
- **Pre-norm**: normalize before each sublayer → stable deep training; OLMo 2 and Gemma 3 add a post-norm inside the residual as well
- **SwiGLU** (Shazeer, 2020): a gated MLP gives better loss per parameter than ReLU/GELU MLPs
- **RoPE** (Su et al., 2021): rotate q and k by position → only the relative offset enters the dot product; no learned table; extends to longer contexts by scaling (YaRN)
- **GQA** (Ainslie et al., 2023): share K,V across groups of heads → KV cache ~8× smaller in Llama-3-70B (64 query heads, 8 KV heads) at negligible loss
- **QK-norm** (Dehghani et al., ViT-22B, 2023): normalize q and k before the dot product → no attention-logit blow-ups; standard in OLMo 2, Gemma 3, Qwen3

RMSNorm

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_i x_i^2 + \epsilon}} \odot g$$

SwiGLU

$$\text{FFN}(x) = (\text{SiLU}(x W_1) \odot x W_3) W_2$$

RoPE: only $n-m$ matters

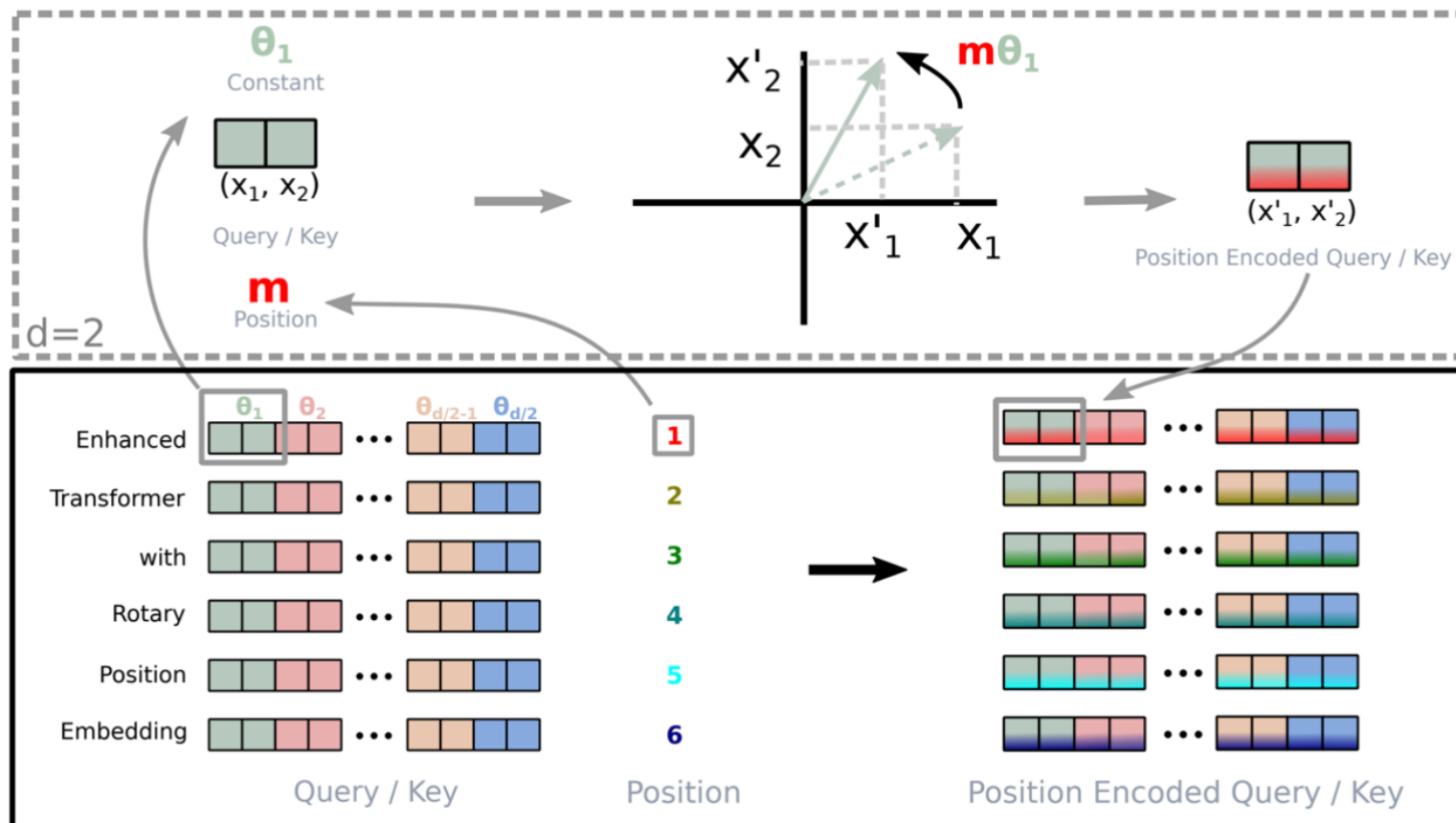
$$\langle R_m q, R_n k \rangle = \langle q, R_{n-m} k \rangle$$

QK-norm

$$\text{softmax}\left(\frac{\hat{q} \hat{k}^\top}{\sqrt{d}}\right) v, \quad \hat{q} = \text{RMSNorm}(q)$$

RoPE: rotary position embeddings

There are many rotations, which one do you pick?

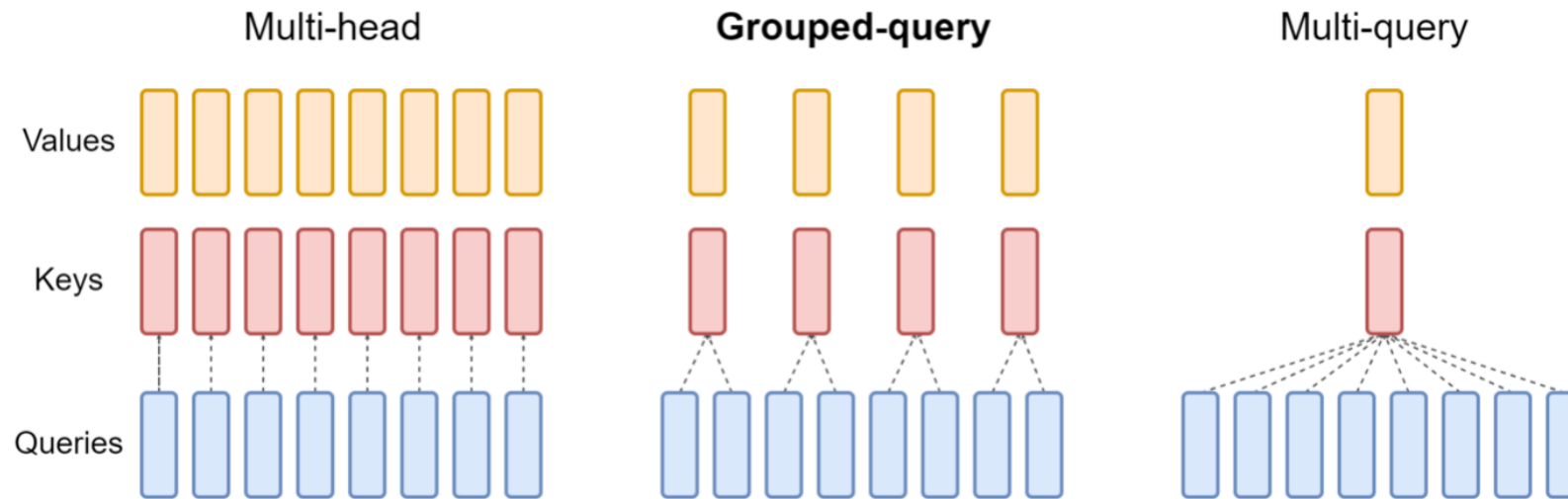


[Su et al 2021]

Just pair up the coordinates and rotate them in 2d (motivation: complex numbers)

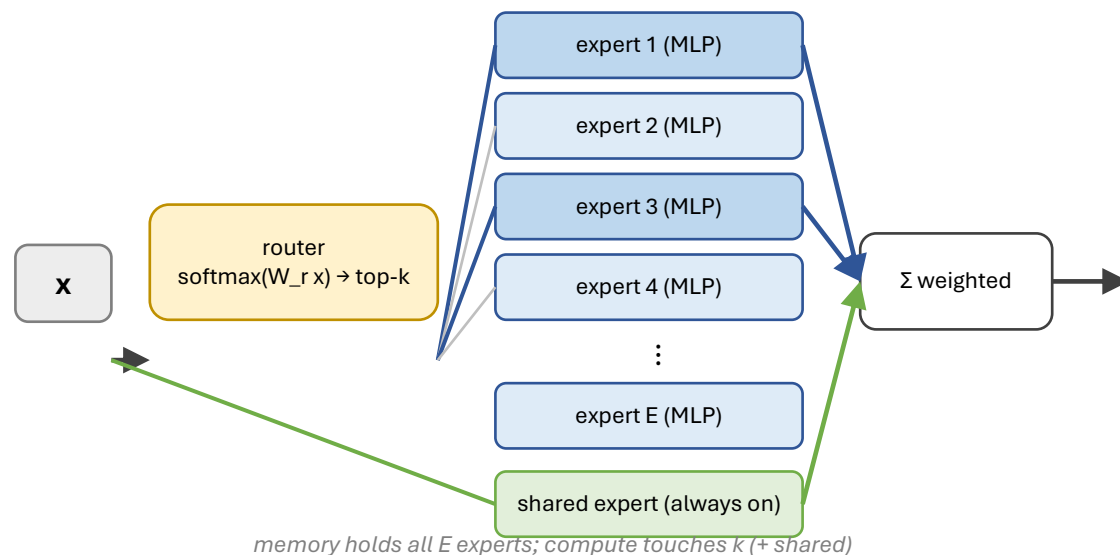
Recent extension – GQA

Don't go all the way to one dimension of KV – have fewer dims



Simple knob to control expressiveness (key-query ratio) and inference efficiency

Mixture of Experts: more parameters, same FLOPs



Model (date)	Total	Active	Experts
DeepSeek-V3 (Dec 2024)	671B	37B	256 + 1 shared, 8 active
Kimi K2 (Jul 2025)	1T	32B	384 + 1 shared, 8 active
Qwen3-235B-A22B (Apr 2025)	235B	22B	128, 8 active
gpt-oss-120B (Aug 2025)	117B	5.1B	128, 4 active

- Router picks the top-k of E expert MLPs per token; output = weighted sum (+ one always-on shared expert in DeepSeek, Kimi)
- All experts sit in memory, only the active ones are computed → 671B parameters but 37B per token (DeepSeek-V3)
- Load balancing: an auxiliary loss (Switch Transformer, 2021) or DeepSeek's auxiliary-loss-free per-expert bias
- Trend: many small 'fine-grained' experts (128–384) instead of a few large ones (Mixtral 8×7B, 2023: 8 experts, top-2)

The “Attention Sink” Phenomenon

- We observed the phenomenon in the SpAtten project in 2021, but wan’t able to explain it until 2023:

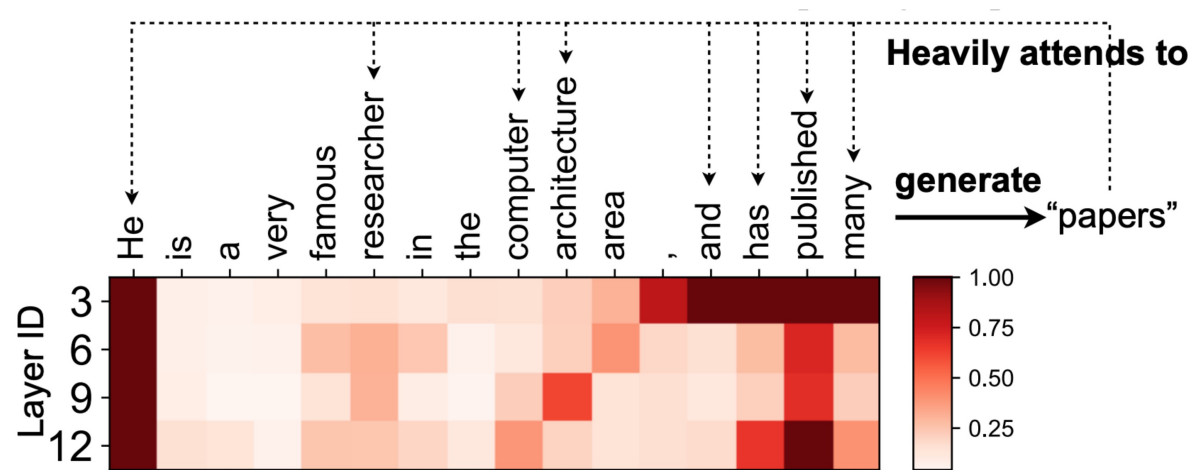


Fig. 23. Cumulative importance scores in GPT-2. Unimportant tokens are pruned on the fly. Important tokens are heavily attended.

GPT-2 for Language Modeling

Du Fu was a great poet of the Tang dynasty. Recently a variety of styles have been used in efforts to translate the work of Du Fu into English

~~Du Fu was a great poet of the Tang dynasty. Recently a variety of styles have been used in efforts to translate the work of Du Fu into English~~

~~Du Fu was a great poet of the Tang dynasty. Recently a variety of styles have been used in efforts to translate the work of Du Fu into English~~

‘English’ is the generated token.